

1 The big picture

Every building block, and how they connect. Knowledge is prepared once (top row); answers are produced on demand (middle row); patient data stays in its own per-practitioner vault. Follow the arrows.

A · PREPARING THE KNOWLEDGE

PORTAL · RUN THE CLINIC

Admin Portal

Uploads sources, sets kind/origin, edits grade and topics after the fact, triggers /api/relink and /api/consolidate maintenance, manages practitioners.

KNOWLEDGE · READABLE

The Library

Every source kept byte-for-byte alongside its passages — searchable by title/topic/kind, graded, with its original file archived for citation review.

INGESTION PIPELINE

Reader · Embedder · Graph-builder

Reads, titles, grades and chunks the source, embeds every passage, and extracts the clinical entities & relationships each passage actually states.

KNOWLEDGE · AI-READY

AI Knowledge Base — Neo4j

Document → Chunk → Entity, one atomic write. Every passage carries its source's 1–10 reliability grade, so relevance and trust stay separate all the way through to the answer.

B · ASKING & ANSWERING

PORTAL · DAILY WORKSPACE

Practitioner Portal

Opens a client's file (labs, history, past sessions), asks the AI team a question scoped to that client, sees every citation and the full traversal path.

THE AI ANSWER TEAM

Answer Engine + Reasoner

Answer Engine

Forms the seed query, then judges relevance hop by hop through the graph.

Reasoner

Studies everything accumulated, writes the grounded answer, cites every claim.

PATIENT DATA

Patient Records

Postgres, a private schema per practitioner (vault_<id>). Conditions, meds, labs, session history — never written into the graph itself.

SAFETY · OPTIONAL

Anti-Hallucination Check

HHEM scores every answer sentence against its cited source. A weak verdict sends the draft back to the Reasoner once, with the unsupported lines named — not a silent pass-through.

PEOPLE

Clients

Cared for by the practitioner. Clients don't chat with the AI directly — their own portal shows their labs, session summaries and wearable data.

WORLD RESEARCH · PLANNED

Live Literature Lookup

A planned third source — PubMed, queried live at answer time — not built yet. Today the Reasoner draws only on the graded library and the patient's own record.

FOUNDATIONS UNDERNEATH

Postgres, schema-per-practitioner isolation

Every AI-team role independently repointable to another provider — no vendor lock-in

Anti-hallucination check is a local classifier (HHEM), not another LLM grading itself

Full audit trail — every ingest, regrade and question logged

Grade-gated retrieval — each practitioner sets their own reliability floor

→ flow of knowledge & questions

①② the two sources every answer draws on today

③ planned, not yet built

□ dashed = optional or planned

2 How the clinic teaches its AI

Before the assistant can help anyone, the library fills its own shelves. An admin uploads a source and everything else happens automatically, in one write.

1

Upload

An admin drops in a source — a PDF, pasted text, or a scraped web page — via POST /api/sources (or the chunked upload flow for anything large).

2

Extract & split

Text is pulled per page from the file, then split into passages — the citable unit — with page numbers preserved for later citation.

3

Read & grade

The Reader titles it, summarises it, tags its topics (reusing an existing shelf wherever one fits), and proposes a 1–10 reliability grade from the strength of the evidence. An admin can revise the grade or topics at any time.

4

Embed & extract the graph

The Embedder embeds every passage; the Graph-builder pulls out the clinical entities and relationships that passage actually states — one failed passage never blanks the rest of the document.

engineers call this: embeddings, held in Neo4j's own vector index — semantic search, not just keyword match.

5

One atomic write

Document, Chunks, and Entities land together in Neo4j — or not at all. The source is immediately searchable (seed search) and traversable (graph expansion) the moment it lands; it never sits as a disconnected island.

🔍 The admin Library shows every source's grade, topics and ingest date, plus the graph's running totals (documents, chunks, entities, typed relationships). An exact re-upload of the same content is blocked automatically (content-hash check) rather than silently duplicated. Each practitioner sets their own reliability floor — nothing below their grade threshold reaches an answer for them.

3 How a question gets answered

A real moment from the consultation room — this is the heart of the product.

1

The question

During a session, the practitioner opens the client's file and asks the AI team a question, scoped to that patient.

2

The Answer Engine listens

It reads the question plus the patient's context and forms the seed search query. It doesn't answer — it gathers.

3

Two sources today

It pulls from the clinic's own graded knowledge base ① (seed search, then LLM-judged graph traversal hop by hop) and the patient's own record ② — labs, history, past sessions, from their practitioner's Postgres vault. A third source — live PubMed lookup — is planned, not built yet.

4

The Reasoner writes the answer

A senior model studies everything accumulated — fresh, every time — and writes the answer using only that material, with inline citations showing exactly where each point comes from.

engineers call this: retrieval-augmented generation (RAG) — grounded through a knowledge graph, not a flat vector search.

5

The anti-hallucination check

Before the answer is shown, HHEM scores every sentence against its cited source. A weak verdict sends the draft back to the Reasoner once, with the unsupported lines named — a clean pass, a bounded retry, or an honestly flagged answer.

checker: HHEM-2.1-Open (vectara/hallucination_evaluation_model), open-source, run locally — not another LLM grading its own kind's work.

OPTIONAL

6

The answer

Back in the chat within seconds, with clickable citations back to the exact passage. The full turn — question, answer, traversal path, check verdict — is saved to the session automatically.

7

It remembers, on request

The practitioner can generate a short session summary at any point — saved straight to the client's record, and explicitly flagged if any answer in that session wasn't a clean check pass.

Meet the AI team

Not one do-everything model, but five specialised roles — each on the model actually suited to that job — plus a sixth, non-LLM teammate that checks every answer before a practitioner sees it. Every role can be independently repointed to a different provider without touching business logic.



INGESTION

The Reader

Reads a new source, writes its title, summary, topics and a suggested 1–10 reliability grade.

READER_MODEL · Qwen/Qwen3-30B-A3B-Instruct-2507



INGESTION

The Graph-builder

Pulls the clinical entities and relationships out of each passage and links them, so related material across the library can find each other.

GRAPH_BUILDER_MODEL · Qwen/Qwen3-235B-A22B-Instruct-2507



INGESTION

The Embedder

Embeds every knowledge-base chunk once, at ingest time, for seed (vector + full-text) search at question time.

EMBEDDER_MODEL · Qwen/Qwen3-Embedding-8B



RETRIEVAL

The Answer Engine

Forms the seed query, then judges which candidates are worth expanding at every hop of graph traversal — never told the grade, so relevance and trust stay decided separately.

RETRIEVAL_MODEL · nvidia/Nemotron-3_5-Lightning



ANSWER

The Reasoner

The senior model — studies the accumulated, graded evidence and the patient's own data, and writes the grounded answer, every claim cited.

REASONER_MODEL · moonshotai/Kimi-K3



OPTIONAL

QUALITY

The Checker

Independently scores each answer sentence against its cited evidence before a practitioner sees it — not a chat model, a dedicated classifier, run locally.

CHECKER_MODEL · vectara/hallucination_evaluation_model (HHEM-2.1-Open)

💡 All five chat-model roles currently run on Nebius's OpenAI-compatible token factory (one shared key, one bill) — but nothing in ingestion, retrieval or reasoning code constructs a client or knows a base_url. Repointing one role to a different provider is a config change, not a code change.